

Buổi 4

Tham lam và đối số đối chỗ, bản Python

Kiến thức, mười bài tập và mười code mẫu

Ngày 23 tháng 9 năm 2026

TS. Đỗ Phúc Hảo

Tài liệu một file cho buổi tham lam, bản Python. Gồm bốn phần: kiến thức, mười đề bài, gợi ý và lời giải, và mười code mẫu đầy đủ.

Đề bài và bộ test dùng chung với bản C++ ở `../../topics/04-greedy/`. Cùng một đề, cùng một bộ test, khác ngôn ngữ. Với buổi này cái khác không nằm ở thuật toán, vì tham lam là mười vòng quét ngắn ở cả hai ngôn ngữ. Cái khác nằm ở **cách sắp xếp**: `sorted` của Python ổn định còn `std::sort` thì không, Python không có tham số comparator, và hàng đợi ưu tiên của hai ngôn ngữ mặc định ngược chiều nhau. Mục I.6, I.7 và I.4 nói thẳng ba chuyện đó bằng số đo.

1. Mục lục

Phần	Nội dung
I	Kiến thức, mười mục
II	Mười đề bài A tới J
III	Gợi ý phân tầng và lời giải từng bài
IV	Mười code mẫu đầy đủ

Phần I. Kiến thức

Mục tiêu: viết được lời giải tham lam, và quan trọng hơn, **biết khi nào nó sai**. Tham lam là họ thuật toán nguy hiểm nhất trong ICPC, không phải vì khó viết mà vì nó luôn chạy được và luôn cho ra một con số trông hợp lý.

Mục	Nội dung	Bài luyện
1	Tham lam là gì và vì sao nó nguy hiểm	tất cả
2	Đối số đổi chỗ, mẫu chứng minh chuẩn	A, D, I
3	Ba họ bài kinh điển	A tới J
4	Tham lam hối tiếc, và heapq là hàng đợi ưu tiên NHỎ NHẤT	F
5	Khi tham lam sai, và cách nhận ra sớm	F
6	sorted của Python ỔN ĐỊNH, std::sort thì không	C, A, F
7	Python không có comparator, và cmp_to_key trả giá bao nhiêu	H
8	Đọc dữ liệu, và con số thật của buổi này	tất cả
9	Cạm bẫy Python ngoài phần thuật toán	tất cả
10	Bộ khung tự hỏi khi gặp đề mới	tất cả

1. Tham lam là gì và vì sao nó nguy hiểm

Tham lam là chiến lược xây lời giải bằng cách ở mỗi bước chọn phương án trông tốt nhất **tại chỗ**, không quay lui, không xét lại.

Đặc điểm khiến nó dễ chịu: code ngắn, thường chỉ một lần sắp xếp cộng một vòng quét, độ phức tạp $O(n \log n)$. Cả mười lời giải của buổi này đều dưới bốn mươi dòng.

Đặc điểm khiến nó nguy hiểm: **không có gì trong code báo cho bạn biết là chiến lược sai**. Quy hoạch động sai thì thường ra kết quả vô lý hoặc chạy quá lâu. Tham lam sai thì ra một con số gần đúng, qua hết test mẫu, và chỉ chết ở test lớn của hệ thống chấm.

Ở Python chuyện này còn kín hơn một bậc. Bên C++ một lời giải tham lam hỏng vẫn có thể lộ ra qua tràn số hoặc hành vi không xác định, tức là qua một triệu chứng không phải kết quả sai. Số nguyên Python không tràn và chỉ số âm không đọc ra ngoài vùng nhớ, nên **triệu chứng duy nhất còn lại là con số in ra sai**. Bộ test là tất cả những gì bạn có.

Vì vậy quy trình bắt buộc cho mọi bài tham lam gồm hai bước, và không được bỏ bước nào:

- Phát biểu chiến lược thành một câu.
- Chứng minh nó đúng, hoặc nếu không chứng minh được thì đối chiếu vét cạn trước khi nộp.

Mười lời giải mẫu trong Phần IV đã đi qua bước hai: 3000 bộ dữ liệu ngẫu nhiên cỡ nhỏ, mười bài mỗi bộ, đối chiếu với vét cạn viết bằng thuật toán khác hẳn (duyet mọi tập con, duyet mọi hoán vị, tìm kiếm theo chiều rộng, ghép cặp đệ quy). Lệnh θ trên 3000.

2. Đối số đổi chỗ, mẫu chứng minh chuẩn

Đây là công cụ chứng minh dùng được cho hầu hết bài tham lam trong ICPC. Mẫu lập luận luôn có cùng hình dạng:

Giả sử tồn tại một lời giải tối ưu **khác** với lời giải tham lam. Xét vị trí đầu tiên mà hai bên khác nhau. Ta chứng minh có thể **đổi chỗ** để lời giải tối ưu giống tham lam ở vị trí đó mà không làm nó tệ đi. Lập lại, ta biến lời giải tối ưu thành lời giải tham lam, nên tham lam cũng tối ưu.

Áp vào bài D, xếp việc để tổng thời gian hoàn thành nhỏ nhất. Chiến lược: làm việc ngắn trước.

Chứng minh: giả sử trong một thứ tự tối ưu có hai việc **kề nhau** với việc dài đứng trước việc ngắn, độ dài lần lượt là a và b với $a > b$. Gọi T là thời điểm bắt đầu của cặp đó.

- Thứ tự hiện tại đóng góp $(T + a) + (T + a + b)$.
- Sau khi đổi chỗ, đóng góp $(T + b) + (T + b + a)$.
- Hiệu là $a - b > 0$, tức là đổi chỗ **giảm** tổng.
- Mọi việc khác không đổi gì, vì tổng thời lượng của cặp không đổi nên các việc phía sau bắt đầu đúng thời điểm cũ.

Vậy không lời giải tối ưu nào chứa cặp kề nhau bị nghịch thế, tức là thứ tự tối ưu phải tăng dần.

Áp vào bài A, chọn nhiều khoảng không giao nhất. Chiến lược: sắp theo đầu mút phải rồi lấy tham lam.

Chứng minh: trong mọi lời giải tối ưu, thay khoảng được chọn đầu tiên bằng khoảng kết thúc sớm nhất. Khoảng mới kết thúc không muộn hơn nên không chặn thêm khoảng nào so với trước, và số khoảng được chọn không đổi. Vậy tồn tại lời giải tối ưu bắt đầu giống hết tham lam, rồi quy nạp cho phần còn lại.

Hai câu này là toàn bộ nội dung cần nắm. Nếu bạn viết được một trong hai lập luận trên cho bài của mình thì lời giải đáng tin. Nếu không viết được thì phải đổi chiều vết cặn.

3. Ba họ bài kinh điển

Họ một: sắp xếp theo một khóa rồi quét. Bài A, B, D, H, I. Toàn bộ độ khó nằm ở việc chọn đúng khóa sắp xếp. Bài A sắp theo đầu mút phải chứ không phải trái, bài D sắp tăng dần chứ không phải giảm, bài H sắp theo một khóa lạ.

Họ hai: quét theo trục và duy trì một đại lượng. Bài C, E, G, J. Ở đây bạn đi từ trái sang phải và luôn giữ một con số như số phòng đang dùng, hoặc điểm xa nhất còn với tới.

Họ ba: tham lam kèm cấu trúc dữ liệu. Bài F. Khi quyết định ở mỗi bước cần biết phần tử nhỏ nhất trong tập đã chọn, hàng đợi ưu tiên vào cuộc.

Ở Python họ một có một tin vui rất cụ thể: bạn gần như không phải viết vòng lặp nào cho phần sắp xếp. `sorted(map(int, data[a:b]))` đọc, ép kiểu và sắp xếp hoàn toàn trong thư viện C. Bài D nhờ đó không còn một vòng lặp Python nào, vì `sum(accumulate(t))` cũng chạy ở mức C.

4. Tham lam hối tiếc, và heapq là hàng đợi ưu tiên NHỎ NHẤT

Bài F là bài hay nhất của buổi, vì nó cho thấy tham lam ngây thơ sai ở đâu và sửa thế nào.

Bài toán: mỗi việc mất đúng một đơn vị thời gian, có hạn chót d và lợi nhuận p . Tối đa tổng lợi nhuận.

Tham lam ngây thơ: sắp theo lợi nhuận giảm dần, nhận việc nào còn chỗ. Cách này đúng nếu cài đúng, nhưng cần cấu trúc tìm khe trống, và nếu bạn cài ẩu thì sai.

Tham lam hối tiếc, đơn giản hơn để cài đúng:

```
job.sort()                # by deadline ascending
taken = []               # min heap of profits
for d, p in job:
    heappush(taken, p)    # take it first, decide later
    total += p
    if len(taken) > d:    # one job too many for this deadline
        total -= heappop(taken) # regret the least profitable choice
```

Ý tưởng: cứ nhận đã, rồi khi phát hiện đã nhận quá số việc mà hạn chót cho phép thì **hối tiếc**, bỏ đi việc có lợi nhuận thấp nhất trong số đã nhận. Bỏ việc thấp nhất luôn không tệ hơn bỏ việc khác, và đó chính là đổi số đổi chỗ ở dạng động.

Và đây là chỗ hai ngôn ngữ ngược chiều nhau, đáng nhớ hơn cả bản thân thuật toán.

	Mặc định	Muốn chiều kia thì làm gì
heapq của Python	hàng đợi nhỏ nhất	đẩy -x vào, lấy ra thì đổi dấu lại
priority_queue của C++	hàng đợi lớn nhất	viết đủ ba tham số với greater<>

Bài này cần hàng đợi nhỏ nhất. Nghĩa là bên C++ bạn phải nhớ viết `priority_queue<ll, vector<ll>, greater<ll>>`, quên là thành hàng đợi lớn nhất và bỏ mất việc tốt nhất; còn bên Python mặc định đã đúng, không phải làm gì cả. Đừng vì thế mà kết luận Python dễ hơn: khi gặp bài cần lớn nhất thì vai trò đảo lại, và trong heapq **không có** tham số nào để đảo chiều. Chỉ có mỗi cách đẩy số đối vào.

Cạm bẫy thứ hai của mục này là một dòng trông rất vô hại:

Viết	Nó làm gì
<code>heappop(taken)</code>	lấy ra phần tử nhỏ nhất , và sửa lại đồng cho đúng bất biến
<code>taken.pop()</code>	lấy ra ô cuối list , một phần tử bất kỳ, và làm hỏng bất biến của đồng

Bất biến của đồng chỉ hứa rằng ô 0 là nhỏ nhất. Ô cuối không hứa gì cả. Đổi `heappop(taken)` thành `taken.pop()` là một lỗi gõ một chữ, chương trình vẫn chạy, vẫn in ra một con số lợi nhuận trông hợp lý.

5. Khi tham lam sai, và cách nhận ra sớm

Ví dụ kinh điển: đổi tiền. Với hệ mệnh giá 1, 3, 4 và số tiền 6, tham lam lấy tờ lớn nhất trước cho ra 4 + 1 + 1 tức ba tờ, trong khi tối ưu là 3 + 3 tức hai tờ. Không có cách sắp xếp nào cứu được, bài này cần quy hoạch động.

Dấu hiệu cảnh báo tham lam có thể sai:

- Lựa chọn ở bước này **ảnh hưởng tới nhiều bước sau theo cách phức tạp**, không chỉ đơn giản là chiếm chỗ.
- Bài có ràng buộc dạng tổng hoặc dạng ba lô, không phải ràng buộc dạng thứ tự hay khoảng.
- Bạn thấy mình phải chia nhiều trường hợp trong hàm chọn.

Cách kiểm rẻ nhất và chắc nhất: **đối chiếu vét cạn**. Sinh dữ liệu với n từ 1 tới 8, so tham lam với duyệt mọi tập con hoặc mọi hoán vị. Phản ví dụ nhỏ nhìn là hiểu ngay.

Python là ngôn ngữ tốt nhất trong hai ngôn ngữ để làm việc này, vì `itertools` cho bạn bản vét cạn gần như miễn phí:

```
import itertools
# every subset of a list of n items
for m in range(1 << n):
    sel = [a[i] for i in range(n) if m >> i & 1]
# every ordering of a list
for perm in itertools.permutations(a):
    ...
```

Một cảnh báo từ chính buổi dựng tài liệu này, và nó đáng giá hơn lời khuyên ở trên. Lần chạy đối chiếu đầu tiên báo bài D lệch 262 trên 300 ca. Cả 262 ca đều là **bộ vét cạn sai**, không phải lời giải sai: dòng chọn giá trị nhỏ nhất trong bản vét cạn bị viết lồng ba tầng điều kiện và trả về nhầm biến. Bài học: một bộ đo vừa viết xong thì cũng chỉ là một chương trình chưa ai kiểm. Khi vét cạn và tham lam lệch nhau, **in ra cả phần**

ví dụ rồi tự tính tay trước khi kết luận bên nào sai. Ở đây tính tay một ca hai phần tử $[4, 8]$ là đủ: đáp án đúng 16, tham lam cho 16, vét cạn cho 12.

6. sorted của Python ỔN ĐỊNH, std::sort thì không

Đây là khác biệt quan trọng nhất của cả buổi, vì tham lam sống bằng sắp xếp.

sorted và list.sort của Python **ổn định**: hai phần tử có khóa bằng nhau giữ nguyên thứ tự xuất hiện ban đầu. std::sort của C++ **không** hứa điều đó, muốn ổn định phải gọi stable_sort.

Nghe thì có vẻ Python an toàn hơn. Thực tế nó nguy hiểm hơn, và lý do rất cụ thể: **tính ổn định che giấu một khóa sắp xếp thiếu tiêu chí phụ**. Bên C++, một chương trình quên tiêu chí phụ sẽ cho kết quả khác nhau giữa các lần biên dịch hoặc các bộ dữ liệu, tức là nó tự tổ cáo. Bên Python nó cho kết quả **nhất quán**, và sự nhất quán đó do một chi tiết ở chỗ khác trong chương trình quyết định chứ không phải do lỗi gọi sort.

Bài C là ví dụ. Cần xử lý sự kiện đóng trước sự kiện mở khi trùng thời điểm. Giả sử ta sắp theo thời gian một mình, `key=lambda e: e[0]`, rồi xây danh sách sự kiện theo ba cách khác nhau từ **cùng một** dữ liệu là hai cuộc họp (0, 5) và (5, 10), đáp án đúng là một phòng:

Cách xây danh sách sự kiện	Kết quả
xen kẽ, mỗi cuộc họp đẩy mở rồi đóng	1, đúng
đẩy hết sự kiện mở trước, rồi tới đóng	2, sai
đẩy hết sự kiện đóng trước, rồi tới mở	1, đúng

Ba dòng này đã chạy thật, không phải suy luận. Cùng một lời gọi sort, cùng một dữ liệu, ba kết quả do thứ tự append quyết định. Một người viết bản thứ ba sẽ nộp bài, được AC, và không bao giờ biết mình đã thiếu một tiêu chí.

Sắp cả tuple (t, d) thì cả ba cách đều ra 1, vì -1 nhỏ hơn +1 nên chính khóa lo việc đó.

Luật rút ra: khóa sắp xếp phải quyết định hết. Nếu bạn phải giải thích kết quả bằng một câu bắt đầu bằng "vì nó được thêm vào trước" thì khóa của bạn còn thiếu.

Lời giải mẫu của bài C đi xa hơn một bước và **gói hai trường vào một số nguyên**:

```
ev = [x * 2 + 1 for x in so[0::2]] # start at l: odd, sorts AFTER an end
ev += [x * 2 for x in so[1::2]]   # end at r: even, sorts FIRST
ev.sort()
```

Làm vậy được hai thứ cùng lúc. Thứ nhất, luật xử lý trùng thời điểm nằm trong **con số**, không nằm trong lời gọi sort và cũng không nằm trong thứ tự append, nên không chi tiết nào ở xa có thể làm nó đổi nghĩa. Thứ hai, nó nhanh hơn, vì sắp một list số nguyên không phải dựng 400000 đối tượng tuple. Đo trên test lớn của bài C, n bằng $2 \cdot 10^5$ nên có 400000 sự kiện:

Cách sắp	Thời gian
sắp 400000 tuple (t, d)	0,126 s
sắp 400000 số nguyên đã gói	0,066 s

Gần gấp đôi, và cái nhanh hơn cũng là cái an toàn hơn. Đó không phải chuyện thường gặp, nên đáng ghi nhớ.

Còn nơi tính ổn định **không** gây hại thì cũng nên biết, để khỏi sợ hãi vô ích. Bài A sắp theo (r, l); nếu chỉ sắp theo r thì trong nhóm cùng r thứ tự là tùy, nhưng nhiều nhất một khoảng trong nhóm đó được chọn nên số đếm không đổi. Bài F sắp theo (d, p); trong nhóm cùng d thì đồng rớt cuộc giữ lại đúng những lợi nhuận lớn nhất bất kể thứ tự nạp. Cách phân biệt hai loại: hỏi xem đáp án có phụ thuộc thứ tự trong nhóm đồng khóa hay không, chứ đừng hỏi khóa trông đã đủ chưa.

7. Python không có comparator, và cmp_to_key trả giá bao nhiêu

Bài H cần thứ tự "x đứng trước y khi chuỗi x + y lớn hơn chuỗi y + x". Bên C++ đó là một lambda hai tham số đưa thẳng vào sort. Bên Python **không có tham số nào như vậy**. sorted chỉ nhận key, tức một hàm một tham số.

Có hai đường đi, và chênh lệch giữa chúng là bài học của bài này.

Đường một, dùng cmp_to_key. Hàm functools.cmp_to_key bọc hàm so sánh của bạn vào một lớp có __lt__, và từ đó mỗi phép so sánh là một lời gọi hàm Python cộng một đối tượng bọc.

Đường hai, dựng một khóa. Hàm key được gọi đúng n lần, còn comparator được gọi cỡ $n \log n$ lần. Với n bằng $2 \cdot 10^5$ đó là chênh lệch giữa hai trăm nghìn và hơn ba triệu lượt vào trình thông dịch.

Khóa đúng cho bài này: **chính token lặp lại tới một độ dài cố định**. Cơ sở là một mệnh đề đúng: $x + y > y + x$ khi và chỉ khi chuỗi lặp vô hạn của x lớn hơn chuỗi lặp vô hạn của y. Hai chuỗi lặp vô hạn, nếu khác nhau, thì khác nhau trong $\text{lcm}(|x|, |y|)$ ký tự đầu. Đề cho a_i tới 10^9 nên không token nào dài quá 10 chữ số, và bội chung nhỏ nhất của hai số không quá 10 thì không vượt 90. Lấy 100 cho chắc:

```
KEY = 100
a.sort(key=lambda s: (s * (KEY // len(s) + 1))[:KEY], reverse=True)
```

Do trên đúng 200000 token của test lớn bài H, sau khi **trộn ngẫu nhiên** dữ liệu:

Cách viết	Thời gian	Số lần so sánh
cmp_to_key	0,295 s	3 258 488
khóa chuỗi lặp	0,061 s	không đếm, sort so chuỗi ở mức C

Chênh 4,8 lần. Hai bản cho ra **cùng một chuỗi kết quả**, đã đối chiếu.

Và đây là phần quan trọng hơn con số trên. Cũng phép đo đó, chạy trên test 09 nguyên bản của kho thay vì bản trộn:

Dữ liệu	cmp_to_key	Số lần so sánh
test 09 nguyên bản	0,025 s	199 999
cùng dữ liệu, đã trộn	0,295 s	3 258 488

Chênh gần 12 lần, cùng một chương trình, cùng một tập số. Lý do: test 09 của kho **đã gần như sắp sẵn theo chiều giảm**, 199 923 trên 199 999 cặp kề nhau là không tăng. Thuật toán sắp xếp của Python tên là Timsort, và nó nhận ra dãy giảm dài rồi lật lại, nên nó chỉ cần $n - 1$ phép so sánh thay vì $n \log n$.

Nghĩa là **test lớn nhất của bài H không hề kiểm chi phí của hàm so sánh**. Nó chạy, nó đúng, và nó không chứng minh điều gì về tốc độ cả. Đây đúng là điều tác giả bản C++ tự ghi lại ở phần ghi chú cuối tài liệu của họ, về hai test cho đáp án -1: một test rơi vào trường hợp suy biến thì gần như không kiểm được gì. Chỉ có điều lần này trường hợp suy biến không nằm ở đáp án mà nằm ở **thứ tự dữ liệu vào**, một chỗ không ai nghĩ tới khi nhìn file test.

Hệ quả thực tế cho người học buổi này: bản cmp_to_key **vẫn qua** được giới hạn 2 giây của chủ đề, trên bộ test hiện có. Nên đừng học nó như một quy tắc "cmp_to_key là TLE". Hãy học nó như thế này: hàm key gọi n lần, comparator gọi $n \log n$ lần, và bộ test bạn đang có chưa chắc hỏi tới sự khác nhau đó.

8. Đọc dữ liệu, và con số thật của buổi này

Khung chuẩn vẫn là khung của cả nhánh:

```
data = sys.stdin.buffer.read().split()
```

Đọc hết đầu vào thành một khối byte rồi tách một lần, sau đó đi bằng con trỏ và ép kiểu khi cần. Cả mười lời giải của buổi này dùng nó.

Nhưng **đừng nói quá về nó**. Đo trên bài A với test lớn nhất của buổi, 200000 dòng và 3,9 MB, giới hạn của chủ đề là 2000 ms:

Cách đọc	Thời gian	Kết luận
<code>input()</code> từng dòng	0,217 s	vẫn đạt, dùng 11 phần trăm giới hạn
<code>sys.stdin.readline()</code> từng dòng	0,130 s	vẫn đạt
<code>sys.stdin.buffer.read().split()</code>	0,108 s	nhanh nhất

Cả ba đều qua. Chênh lệch giữa cái chậm nhất và cái nhanh nhất là **2 lần**, không phải một bậc. Câu "dùng `input()` là TLE" là một câu sai với chủ đề này.

Vậy vì sao vẫn bắt buộc dùng khối? Vì hai lý do không phải tốc độ trên chính bộ test này. Thứ nhất, khoản 2 lần ấy cộng vào những bài mà phần tính toán đã nặng sẵn, và bạn không muốn phải quay lại đổi cách đọc lúc đã hết giờ. Thứ hai, `read().split()` cắt theo **mọi** khoảng trắng kể cả xuống dòng, nên cách trình bày dữ liệu không làm đổi kết quả; bản đọc từng dòng vỡ ngay khi ai đó viết `n` và dãy trên cùng một dòng.

Chiều ngược lại cũng cần nói cho đủ. Một câu hay gặp là "Python chậm gấp k lần C++, cứ nhân giới hạn lên". Nhánh này **không thể** đo được tỉ lệ đó cho chủ đề 04, vì trên máy dựng tài liệu, trình biên dịch C++ không dựng nổi lời giải mẫu nào của nhánh kia: clang của macOS không có `<bits/stdc++.h>`. Nên ở đây không có bảng so C++ với Python, và sẽ không có cho tới khi ai đó đo thật. Những gì chủ đề 06 đo được cho thấy tỉ lệ ấy đi cả hai chiều, từ Python nhanh hơn 1,6 lần tới chậm hơn 4,4 lần tùy bài, nên một hằng số nhân là thứ không tồn tại.

Con số **có** đo được của chủ đề này: bài chậm nhất trong mười bài là bài F, và nó nằm trong khoảng 121 tới 139 ms qua nhiều lượt đo, tức dưới 7 phần trăm của giới hạn 2000 ms. Cả mười bài đều dưới ngưỡng đó. Dải 121 tới 139 không phải phép đo cẩu thả: cùng một máy, cùng một lệnh, con số dao động theo việc máy đang chạy gì khác. Đó là lý do giới hạn đặt xa số đo chứ không sát nó. Buổi tham lam là buổi nhẹ nhất về tính toán trong cả kho, vì mỗi bài chỉ có một lần sắp xếp và một vòng quét.

9. Cạm bẫy Python ngoài phần thuật toán

Cạm bẫy	Hậu quả
<code>/</code> thay <code>//</code>	<code>/</code> luôn cho số thực, và số thực mất chính xác từ khoảng $9 \cdot 10^{15}$. Đáp án bài D đạt $7 \cdot 10^{14}$ và bài E có vị trí tới 10^{18} , cả hai nằm trong tầm nguy hiểm
<code>taken.pop()</code> thay <code>heappop(taken)</code>	lấy nhầm phần tử và làm hỏng bất biến của đồng, xem mục I.4
tưởng <code>heapq</code> có hàng đợi lớn nhất	không có; muốn lớn nhất thì đẩy số đối
<code>sorted</code> ổn định nên "chắc là đúng"	xem mục I.6, sự nhất quán không phải sự đúng đắn
<code>list.remove(x)</code> để bỏ một phần tử khỏi đồng	$O(n)$ mỗi lần, và phá bất biến
ghép chuỗi kết quả bằng <code>s += ...</code> trong vòng lặp	$O(n^2)$ vì chuỗi Python bất biến; gom list rồi <code>join</code>
gọi <code>max(a, b)</code> trong vòng lặp nóng	lời gọi hàm đắt; viết <code>a if a > b else b</code>
giải mã bytes sang str khi không cần	bài H giữ nguyên bytes từ đầu tới cuối, vì bytes so sánh theo từng byte và với chữ số đó đúng là thứ tự cần

Hai cạm bẫy **biến mất** khi sang Python, và đáng nói ra để khỏi đi tìm:

- **Tổng tràn kiểu 32 bit.** Bản C++ liệt kê đây là cạm bẫy số 8 và nó có thật ở bài D với 10^{15} cùng bài F với $2 \cdot 10^{14}$. Số nguyên Python không tràn, nên cạm bẫy này không tồn tại trong nhánh này. Đừng chép cảnh báo ấy sang.
- **Comparator không bắc cầu làm hàm sắp xếp đọc ra ngoài vùng nhớ.** Bản C++ cảnh báo đúng cho `libstdc++`. Trong Python, một khóa hay một `cmp_to_key` không nhất quán chỉ cho ra thứ tự sai, không

làm hỏng bộ nhớ. Quan hệ của bài H bắc cầu thật, nhưng nếu bạn tự chế một quan hệ khác thì triệu chứng bên này là kết quả sai chứ không phải chương trình chết.

10. Bộ khung tự hỏi khi gặp đề mới

- Chiến lược tham lam của tôi phát biểu thành câu gì?
- Tôi viết được đối số đối chỗ cho nó không? Nếu có, viết một câu ra giấy.
- Nếu không, tôi đã đối chiếu vết cạn với `itertools` chưa?
- Khóa sắp xếp của tôi có quyết định hết không, hay còn chỗ nào phải giải thích bằng "vì nó được thêm vào trước"?
- Có nhóm phần tử nào đồng khóa không? Đáp án có đôi khi đảo thứ tự trong nhóm đó không?
- Tôi cần phần tử nhỏ nhất hay lớn nhất? Nếu lớn nhất thì đã nhớ đẩy số đối vào `heapq` chưa?
- Có trường hợp không tồn tại lời giải không, và tôi in gì khi đó? Vòng lặp của tôi có thoát ra trong trường hợp đó không, hay nó treo?
- Có chỗ nào tôi viết `/` mà ý là `//` không?

Kế tiếp: sang **Phần II** làm bài. Làm xong hãy tự chấm bằng `chạy.py` trước khi mở **Phần III**.

Phần II. Mười bài tập

Đề PDF	Ở đây	Kỹ thuật chính	File nộp
A	Bài 1	sắp theo đầu mút phải	bai01.py
B	Bài 2	hai con trỏ trên hai dãy đã sắp	bai02.py
C	Bài 3	quét sự kiện, khóa gói hai trường	bai03.py
D	Bài 4	đổi số đổi chỗ, accumulate	bai04.py
E	Bài 5	quét và giữ điểm xa nhất	bai05.py
F	Bài 6	tham lam hồi tiếc kèm heapq	bai06.py
G	Bài 7	phủ đoạn bằng ít khoảng nhất	bai07.py
H	Bài 8	khóa sắp xếp thay cho comparator	bai08.py
I	Bài 9	ghép hai đầu	bai09.py
J	Bài 10	tham lam theo tầng	bai10.py

Giới hạn thời gian **2 giây** cho mọi bài. Con số ấy đặt theo số đo, không phải nhân giới hạn của nhánh C++ với một hằng số: bài chậm nhất trong mười bản mẫu đo được 121 tới 139 ms tùy lượt chạy, tức dưới 7 phần trăm giới hạn. Tự chấm:

```
python3 chay.py topics/04-greedy/solutions/bai01.py
python3 chay.py topics/04-greedy --tat-ca
```

Bài A. Chọn nhiều hoạt động nhất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n hoạt động, hoạt động thứ i diễn ra trong khoảng thời gian từ l_i tới r_i . Bạn chỉ tham gia được một hoạt động tại một thời điểm, nhưng được phép rời hoạt động này đúng lúc bắt đầu hoạt động kia. Hãy chọn nhiều hoạt động nhất có thể.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- n dòng tiếp theo, mỗi dòng gồm l_i và r_i ($0 \leq l_i < r_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất là số hoạt động nhiều nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 1 3 2 5 4 7 6 8	2

Giải thích

Chọn hoạt động 1 3 và 4 7. Không có cách nào chọn được ba hoạt động.

Lưu ý

Khóa sắp xếp là toàn bộ độ khó của bài này. Hãy thử nghĩ xem trong các hoạt động còn chọn được thì chọn cái nào sẽ để lại nhiều cơ hội nhất cho phần sau. Riêng với Python: hai cột l và r lấy ra bằng lát cắt `data[0:2]` và `data[1:2]`, không cần vòng lặp nào.

Bài B. Phát bánh

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n đứa trẻ, đứa thứ i chỉ hài lòng nếu nhận được cái bánh có kích thước ít nhất g_i . Có m cái bánh, cái thứ j có kích thước s_j . Mỗi đứa trẻ nhận nhiều nhất một cái bánh và mỗi cái bánh phát cho nhiều nhất một đứa trẻ.

Hãy làm cho số trẻ hài lòng nhiều nhất có thể.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n, m \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên g_i ($1 \leq g_i \leq 10^9$)
- Dòng 3: m số nguyên s_j ($1 \leq s_j \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
3 2 1 2 3 1 2	2

Giải thích

Phát bánh kích thước 1 cho đứa trẻ có nhu cầu 1, và bánh kích thước 2 cho đứa có nhu cầu 2. Đứa có nhu cầu 3 không được phục vụ.

Lưu ý

Với đứa trẻ dễ tính nhất còn lại, nên phát cái bánh nào trong số các bánh còn đủ lớn? Và khi cái bánh đang xét quá nhỏ cho cả đứa dễ tính nhất, con trẻ nào được phép nhích?

Bài C. Số phòng họp tối thiểu

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n cuộc họp, cuộc thứ i bắt đầu lúc l_i và kết thúc lúc r_i . Một phòng chỉ chứa được một cuộc họp tại một thời điểm, nhưng phòng vừa trống lúc t dùng ngay được cho cuộc họp bắt đầu lúc t .

Hãy tìm số phòng tối thiểu để tổ chức được toàn bộ các cuộc họp.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- n dòng tiếp theo, mỗi dòng gồm l_i và r_i ($0 \leq l_i < r_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
3 0 30 5 10 15 20	2

Giải thích

Cuộc họp 0 30 chồng lên cả hai cuộc còn lại, nhưng 5 10 và 15 20 không chồng lên nhau nên hai phòng là đủ.

Lưu ý

Câu về phòng vừa trống trong đề không phải để trang trí. Nó quyết định thứ tự xử lý hai sự kiện xảy ra tại cùng một thời điểm. Riêng với Python: `sorted` ổn định, nên nếu khóa của bạn chỉ có thời gian thì đáp án sẽ do **thứ tự bạn thêm sự kiện vào list** quyết định chứ không do lời gọi `sort`. Đọc mục I.6 trước khi tin vào bản đầu tiên bạn viết.

Bài D. Xếp việc để tổng thời gian hoàn thành nhỏ nhất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n công việc, việc thứ i cần t_i đơn vị thời gian. Chỉ có một máy nên các việc phải làm lần lượt. Thời gian hoàn thành của một việc là thời điểm nó làm xong tính từ lúc bắt đầu.

Hãy chọn thứ tự để **tổng thời gian hoàn thành của tất cả các việc** là nhỏ nhất, và in ra tổng đó.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên t_i ($1 \leq t_i \leq 10^5$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
3 3 1 2	10

Giải thích

Làm theo thứ tự 1, 2, 3 thì các việc hoàn thành lúc 1, 3 và 6, tổng là 10. Mọi thứ tự khác đều cho tổng lớn hơn.

Lưu ý

Đáp án đạt tới $7 \cdot 10^{14}$. Bên C++ đó là cận bão hòa kiểu; bên Python số nguyên không tràn nên cận bão hòa duy nhất còn lại ở chỗ ấy là viết / khi ý là //, vì số thực mất chính xác đúng trong tầm này. Đây cũng là bài mẫu để tập viết đối số đối chỗ, nên hãy thử chứng minh chiến lược của bạn bằng cách xét hai việc kề nhau.

Bài E. Đổ xăng ít lần nhất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Bạn lái xe từ vị trí 0 tới vị trí D trên một đường thẳng. Bình xăng đầy cho phép đi được đúng C đơn vị khoảng cách, và lúc xuất phát bình đang đầy. Trên đường có n trạm xăng tại các vị trí x_1 tới x_n , ở mỗi trạm bạn được đổ đầy bình.

Hãy tìm số lần đổ xăng ít nhất để tới được D, hoặc in -1 nếu không thể tới nơi.

Dữ liệu vào

- Dòng 1: D, C và n ($1 \leq C \leq D \leq 10^{18}$, $1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên x_i ($0 < x_i < D$), không nhất thiết đã sắp xếp

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
100 30 3 25 50 80	3

Giải thích

Từ 0 tới được 30, đổ tại 25 rồi tới 55, đổ tại 50 rồi tới 80, đổ tại 80 rồi tới 110 nên qua đích.

Lưu ý

Khi buộc phải dừng, bạn nên đổ tại trạm nào trong số các trạm đã đi qua? Bên C++ đề này còn bắt bạn để ý D tới 10^{18} nên mọi biến vị trí phải là 64 bit; bên Python chuyện đó tự lo, miễn là không có phép / nào lọt vào.

Bài F. Chọn việc có hạn chót

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n công việc, việc thứ i có hạn chót d_i và lợi nhuận p_i . Mỗi việc mất đúng một đơn vị thời gian và chỉ có một máy, nên tại mỗi đơn vị thời gian 1, 2, 3 và tiếp theo bạn làm nhiều nhất một việc. Việc i chỉ tính lợi nhuận nếu nó được hoàn thành tại một thời điểm không muộn hơn d_i .

Hãy chọn tập việc để tổng lợi nhuận lớn nhất.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- n dòng tiếp theo, mỗi dòng gồm d_i và p_i ($1 \leq d_i \leq n$, $1 \leq p_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 1 20 2 10 1 40 2 30	70

Giải thích

Làm việc có lợi nhuận 40 tại thời điểm 1 và việc có lợi nhuận 30 tại thời điểm 2.

Lưu ý

Hãy thử chiến lược *cứ lấy việc lợi nhuận cao nhất trước* và tự chạy tay trên ví dụ này trước khi tin vào nó. Nếu bạn nhận ra chiến lược đó khó cài đúng, hãy thử hướng ngược lại: duyệt theo hạn chót và cho phép mình đổi ý. Riêng với Python: `heapq` là hàng đợi ưu tiên **nhỏ nhất**, đúng chiều bài này cần, và `heappop(taken)` không phải `taken.pop()`.

Bài G. Phủ đoạn bằng ít khoảng nhất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho đoạn $[0, M]$ và n khoảng, khoảng thứ i là $[l_i, r_i]$. Hãy chọn ít khoảng nhất sao cho hợp của chúng phủ kín toàn bộ đoạn $[0, M]$, hoặc in -1 nếu không phủ kín được.

Dữ liệu vào

- Dòng 1: M và n ($1 \leq M \leq 10^9, 1 \leq n \leq 2 \cdot 10^5$)
- n dòng tiếp theo, mỗi dòng gồm l_i và r_i ($0 \leq l_i \leq r_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 3 0 2 1 4 3 5	3

Giải thích

Phải dùng cả ba khoảng. Hai khoảng bất kỳ đều để hở một phần của đoạn.

Lưu ý

Đừng quên trường hợp không phủ kín được, và hãy bảo đảm vòng lặp của bạn thoát ra trong trường hợp đó thay vì chạy mãi. Một vòng lặp treo ở Python không báo lỗi gì, bộ chấm chỉ kết luận TLE, nên triệu chứng trông giống hết một thuật toán quá chậm.

Bài H. Ghép thành số lớn nhất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n số nguyên dương. Hãy sắp xếp chúng theo một thứ tự nào đó rồi viết liền nhau thành một chuỗi, sao cho chuỗi thu được biểu diễn số lớn nhất có thể.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một dòng duy nhất chứa chuỗi kết quả.

Ví dụ

Dữ liệu vào	Kết quả
5 3 30 34 5 9	9534330

Giải thích

Thứ tự đúng là 9, 5, 34, 3, 30.

Lưu ý

So sánh theo giá trị hoặc theo độ dài đều cho kết quả sai; hãy thử với hai số 9 và 34. Riêng với Python: `sorted` **không có** tham số `comparator`. `functools.cmp_to_key` có tồn tại, nhưng nó biến mỗi phép so sánh thành một lời gọi hàm Python, trong khi một hàm `key` chỉ được gọi n lần. Đọc mục I.7 để biết khoảng cách đo được giữa hai đường.

Bài I. Thuyền cứu hộ

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Có n người cần sơ tán, người thứ i nặng w_i . Mỗi chiếc thuyền chở được nhiều nhất hai người và tổng cân nặng trên thuyền không được vượt quá $limit$.

Hãy tìm số thuyền ít nhất. Để bảo đảm mọi w_i đều không vượt quá $limit$.

Dữ liệu vào

- Dòng 1: n và $limit$ ($1 \leq n \leq 2 \cdot 10^5, 1 \leq limit \leq 10^9$)
- Dòng 2: n số nguyên w_i ($1 \leq w_i \leq limit$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 3 3 2 2 1	3

Giải thích

Một thuyền chở người nặng 3, một thuyền chở cặp 2 và 1, một thuyền chở người nặng 2 còn lại.

Lưu ý

Người nặng nhất chắc chắn phải đi trong chuyến này. Câu hỏi duy nhất là có ai đi cùng được hay không. Hãy để ý phép lùi chỉ số phải nằm ở đâu so với câu điều kiện: đặt sai chỗ thì vòng lặp không tiến và chương trình treo ngay ở ví dụ trong đề.

Bài J. Nhảy ít bước nhất

Giới hạn thời gian: 2 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên không âm. Bạn đứng ở vị trí 1. Đứng ở vị trí i , bạn nhảy được tới bất kỳ vị trí nào trong đoạn từ $i + 1$ tới $i + a_i$.

Hãy tìm số bước nhảy ít nhất để tới vị trí n , hoặc in -1 nếu không tới được.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($0 \leq a_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất. Nếu n bằng 1 thì đáp án là 0.

Ví dụ

Dữ liệu vào	Kết quả
5 2 3 1 1 4	2

Giải thích

Từ vị trí 1 nhảy tới vị trí 2, rồi từ vị trí 2 nhảy thẳng tới vị trí 5.

Lưu ý

Tập các vị trí tới được sau đúng k bước là một đoạn liên chứ không phải một tập rời rạc. Nhận ra điều đó là xong bài, và bạn không cần tới tìm kiếm theo chiều rộng. Đây cũng là bài duy nhất của buổi mà phần việc thật nằm trong một vòng lặp Python, nên giữ thân vòng lặp phẳng: không gọi `max`, không tách hàm con.

Phần III. Gợi ý và lời giải

Đọc từng tầng một, dừng lại ngay khi đã nghĩ ra. Code đầy đủ nằm ở Phần IV.

Bài	Cái bẫy thật sự	Riêng với Python
A	sắp theo đầu mút trái thay vì phải	lấy hai cột bằng lát cắt, không vòng lặp
B	bánh quá nhỏ vẫn bị tính là đã phát	không có
C	sự kiện kết thúc xử lý sau sự kiện bắt đầu	sorted ổn định nên khóa thiếu vẫn nhất quán, xem I.6
D	sắp giảm dần thay vì tăng dần	sum(accumulate(t)), và tràn kiểu không còn là bẫy
E	đổ tại trạm gần nhất thay vì trạm xa nhất trong tầm	không có
F	tham lam ngây thơ, không biết hối tiếc	heappop không phải list.pop, xem I.4
G	chọn khoảng vươn xa nhất bị thay bằng khoảng gấp sau cùng	vòng treo chỉ hiện ra dưới dạng TLE
H	so sánh hai số thay vì so hai cách nối	không có comparator, phải dựng khóa, xem I.7
I	người nặng nhất không rời thuyền khi không ghép được	không có
J	sang tầng mới quá sớm	vòng lặp Python thật, giữ thân vòng phẳng

Bài A. Chọn nhiều hoạt động nhất

Gợi ý 1. Trong các hoạt động còn chọn được, chọn cái nào thì để lại nhiều cơ hội nhất cho phần sau?

Gợi ý 2. Cái **kết thúc sớm nhất**, vì nó giải phóng thời gian sớm nhất. Độ dài hay thời điểm bắt đầu đều không phải khóa đúng.

Gợi ý 3. Sắp theo đầu mút phải, rồi quét và lấy hoạt động nào bắt đầu không sớm hơn thời điểm kết thúc gần nhất đã lấy.

Lời giải. Ghép mỗi hoạt động thành cặp (r, l) rồi sorted, thế là thứ tự tuple mặc định đã sắp theo đầu mút phải. Quét một lượt, giữ biến last là điểm kết thúc của hoạt động vừa lấy. Độ phức tạp $O(n \log n)$.

Đối số đổi chỗ. Trong mọi lời giải tối ưu, thay hoạt động được chọn đầu tiên bằng hoạt động kết thúc sớm nhất. Hoạt động mới kết thúc không muộn hơn nên không chặn thêm gì so với trước, và số hoạt động không đổi. Vậy tồn tại lời giải tối ưu bắt đầu giống tham lam, rồi quy nạp cho phần còn lại.

Sắp theo đầu mút **trái** là sai, và sai rất dễ thấy trên ví dụ: một hoạt động dài bắt đầu sớm sẽ nuốt mất nhiều hoạt động ngắn phía sau. Nhưng code thì vẫn chạy trơn tru, nên nếu bạn không thử ví dụ đó thì không có gì báo cho bạn biết.

Python. Hai cột dữ liệu lấy ra bằng lát cắt `so[0::2]` và `so[1::2]`, rồi `sorted(zip(...))`. Cả ba bước chạy trong thư viện C, nên vòng lặp Python duy nhất của bài là vòng quét tham lam.

Bài B. Phát bánh

Gợi ý 1. Sắp cả hai dãy rồi đi hai con trỏ.

Gợi ý 2. Với đĩa trẻ để tính nhất còn lại, nên phát cái bánh nào?

Gợi ý 3. Cái bánh **nhỏ nhất còn đủ**. Dùng bánh to cho nhu cầu nhỏ chỉ làm phí bánh to.

Lời giải.

```
i = j = served = 0
while i < n and j < m:
    if s[j] >= g[i]:
        served += 1
        i += 1
        j += 1
    else:
        j += 1          # this cookie is too small for everyone left
```

Chú ý nhánh else chỉ tăng j. Bánh này nhỏ hơn nhu cầu nhỏ nhất còn lại nên nó vô dụng với mọi đứa trẻ còn lại, bỏ đi là đúng. Nếu bạn tăng cả i trong nhánh đó thì bạn đang bỏ rơi một đứa trẻ mà lẽ ra nó vẫn còn cơ hội với cái bánh sau.

Python. `sorted(map(int, data[a:b]))` làm cả ba việc đọc, ép kiểu và sắp xếp trong một biểu thức. Đừng viết vòng while gọi `int()` từng token.

Bài C. Số phòng họp tối thiểu

Gợi ý 1. Số phòng cần đúng bằng số cuộc họp chồng nhau nhiều nhất tại một thời điểm nào đó.

Gợi ý 2. Biến mỗi cuộc họp thành hai sự kiện, một sự kiện mở và một sự kiện đóng, rồi quét theo thời gian.

Gợi ý 3. Tại cùng một thời điểm, xử lý sự kiện nào trước? Phòng vừa trống lúc t dùng ngay được cho cuộc họp bắt đầu lúc t.

Lời giải. Gộp cả thời gian lẫn loại sự kiện vào **một số nguyên**:

```
ev = [x * 2 + 1 for x in so[0::2]]    # start at l: odd, sorts AFTER an end
ev += [x * 2 for x in so[1::2]]      # end at r: even, sorts FIRST
ev.sort()
```

Rồi quét: gặp số lẻ thì tăng, gặp số chẵn thì giảm, và ghi lại giá trị lớn nhất.

Thứ tự khi trùng thời điểm là toàn bộ cái bẫy, và ở Python nó còn một tầng nữa so với C++. Bên C++, sắp cặp (t, d) thì -1 đứng trước +1 nên sort tự lo. Bên Python điều đó cũng đúng, nhưng nếu bạn lỡ viết `key=lambda e: e[0]` thì sort **ổn định** sẽ giữ nguyên thứ tự thêm vào, và đáp án của bạn phụ thuộc vào một chi tiết nằm cách đó mười dòng. Đã đo trên hai cuộc họp (0, 5) và (5, 10), đáp án đúng là 1:

Cách xây danh sách sự kiện	Kết quả
xen kẽ, mỗi cuộc họp đẩy mở rồi đóng	1
đẩy hết sự kiện mở trước	2
đẩy hết sự kiện đóng trước	1

Gói vào số nguyên thì cả ba cách đều ra 1, vì luật nằm trong chính con số. Nó còn nhanh hơn: sắp 400000 số nguyên mất 0,066 s, sắp 400000 tuple mất 0,126 s.

Bài D. Xếp việc để tổng thời gian hoàn thành nhỏ nhất

Gợi ý 1. Một việc đứng càng sớm thì thời lượng của nó bị cộng vào tổng càng nhiều lần.

Gợi ý 2. Việc thứ i trong thứ tự đóng góp `t_i` nhân với số việc từ nó trở đi. Vậy việc dài nên đứng đầu?

Gợi ý 3. Sắp tăng dần.

Lời giải. Sắp tăng dần rồi cộng dồn, $O(n \log n)$.

Đổi số đổi chỗ, viết đầy đủ vì đây là bài mẫu để dạy. Giả sử thứ tự tối ưu có hai việc kề nhau, việc dài a đứng trước việc ngắn b , với $a > b$, và cặp đó bắt đầu tại thời điểm T .

- Đóng góp hiện tại: $(T + a) + (T + a + b)$.
- Sau khi đổi chỗ: $(T + b) + (T + b + a)$.
- Hiệu bằng $a - b > 0$, tức là đổi chỗ làm tổng **giảm**.
- Các việc khác không bị ảnh hưởng, vì tổng thời lượng của cặp không đổi.

Vậy không lời giải tối ưu nào chứa cặp kề nhau bị nghịch thế, nên thứ tự tối ưu phải tăng dần.

Python. Cả bài gọn trong hai dòng, vì `itertools.accumulate` sinh mọi tổng tiền tố và `sum` cộng chúng lại, cả hai ở mức C :

```
t = sorted(map(int, data[1:1 + n]))
print(sum(accumulate(t)))
```

Cái bẫy còn lại của bản C++ là kiểu dữ liệu, vì tổng đạt $7 \cdot 10^{14}$. Bên Python bẫy đó không tồn tại. Cái thay chỗ nó là dấu `:` nếu ở đâu đó bạn viết `/` thay `//`, kết quả thành số thực và số thực mất chính xác từ khoảng $9 \cdot 10^{15}$, tức là ngay sát tầm này.

Bài E. Đồ xăng ít lần nhất

Gợi ý 1. Đi càng xa càng tốt trước khi buộc phải dừng.

Gợi ý 2. Khi buộc phải đổ, nên đổ tại trạm nào trong số các trạm đã đi qua?

Gợi ý 3. Trạm **xa nhất** trong tầm với, vì nó đẩy điểm tới xa nhất.

Lời giải.

```
while reach < D:
    best = -1
    while i < n and x[i] <= reach:
        best = x[i]          # keep going: the last one drained is the farthest
        i += 1
    if best < 0:
        print(-1)
        return
    stops += 1
    reach = best + C
```

Chú ý vòng `while` bên trong quét hết mọi trạm trong tầm chứ không lấy trạm đầu tiên. Đổi nó thành `i` là bạn đổ xăng tại từng trạm một, số lần đổ tăng vọt, mà kết quả vẫn là một con số hợp lý nên rất khó nghĩ.

Chỉ số `i` không bao giờ lùi nên tổng chi phí là $O(n)$ sau khi sắp xếp, giống hết lối phân tích khâu hao ở buổi 2.

Python. Đề cho D tới 10^{18} . Bên C++ đó là dòng cảnh báo phải dùng 64 bit; bên Python số nguyên dài bao nhiêu cũng được, nên dòng cảnh báo tương ứng ở đây chỉ còn là: đừng để phép `/` nào lọt vào biểu thức vị trí.

Bài F. Chọn việc có hạn chót

Gợi ý 1. Thử chiến lược "lấy việc lợi nhuận cao nhất trước" trên ví dụ trong đề. Nó có ra 70 không?

Gợi ý 2. Hãy đổi hướng: duyệt theo hạn chót tăng dần, và **cứ nhận việc đã**.

Gợi ý 3. Khi số việc đã nhận vượt quá hạn chót hiện tại, bỏ đi việc có lợi nhuận nhỏ nhất trong số đã nhận. Cần một cấu trúc cho ra phần tử nhỏ nhất, tức là đồng nhỏ nhất.

Lời giải.

```

job = sorted(zip(so[0::2], so[1::2])) # (deadline, profit), deadline ascending
taken = []
for d, p in job:
    heappush(taken, p)
    total += p
    if len(taken) > d:
        total -= heappop(taken)

```

Đây là **tham lam hồi tiếc**: nhận trước, và khi phát hiện đã nhận quá số việc mà hạn chót cho phép thì hồi tiếc, bỏ đi lựa chọn kém nhất. Bỏ việc có lợi nhuận nhỏ nhất luôn không tệ hơn bỏ việc khác, và đó chính là đối số đối chỗ ở dạng động.

Ba chỗ dễ sai, và chỗ thứ hai đổi hẳn hình dạng khi sang Python:

- **Bỏ nhầm việc.** Phải bỏ phần tử nhỏ nhất của toàn bộ tập đã nhận, không phải bỏ việc p vừa nhận. Nếu bạn luôn bỏ việc vừa nhận thì thuật toán thoái hóa thành nhận theo thứ tự hạn chót, và mất hết ý nghĩa.
- **Chiều của hàng đợi ưu tiên.** Bên C++ đây là `priority_queue` mặc định là lớn nhất, phải viết đủ ba tham số với `greater<>` mới thành nhỏ nhất. Bên Python `heapq` mặc định đã là nhỏ nhất, đúng chiều cần. Nhưng đừng đọc thành "Python an toàn hơn": khi bài cần lớn nhất thì `heapq` **không có** tham số nào để đảo chiều, bạn buộc phải đẩy $-p$ vào rồi đổi dấu lúc lấy ra.
- **Lấy nhầm bằng ô cuối list.** `taken.pop()` không phải `heappop(taken)`. Đồng chỉ hứa ô 0 là nhỏ nhất; ô cuối list không hứa gì. Gỡ nhầm một chữ ở đây cho ra một con số lợi nhuận trông hoàn toàn hợp lý.

Cạm bẫy thứ ba của bản C++, tổng lợi nhuận tràn `int` ở mức $2 \cdot 10^{14}$, không còn tồn tại bên Python.

Bài G. Phủ đoạn bằng ít khoảng nhất

Gợi ý 1. Bạn đang đứng ở điểm đã phủ tới, gọi là `covered`. Khoảng nào dùng được?

Gợi ý 2. Mọi khoảng có $l \leq \text{covered}$. Trong số đó chọn cái nào?

Gợi ý 3. Cái có r lớn nhất, vì nó đầy `covered` đi xa nhất.

Lời giải.

```

while covered < M:
    best = covered
    while i < n and v[i][0] <= covered:
        r = v[i][1]
        if r > best:
            best = r
        i += 1
    if best == covered: # no progress possible
        print(-1)
        return
    covered = best
    used += 1

```

Chú ý điều kiện dừng khi không tiến thêm được. Thiếu nó thì vòng lặp treo vĩnh viễn ở những dữ liệu không phủ kín được, và ở Python một vòng treo không ném ngoại lệ nào cả: bộ chấm chỉ kết luận TLE, nên triệu chứng trông y hệt một thuật toán quá chậm. Hai nguyên nhân khác hẳn nhau mà cùng một dòng kết luận.

Giữ lại khoảng **gấp sau cùng** thay vì khoảng **vươn xa nhất** là lỗi hay gặp khi gỡ nhanh, và nó cho đáp án đúng trên mọi dữ liệu mà các khoảng tình cờ đã sắp theo đầu mút phải.

Python. Viết `if r > best: best = r` chứ đừng `best = max(best, r)`: `max` là một lời gọi hàm thật, còn phép so sánh thì không.

Bài H. Ghép thành số lớn nhất

Gợi ý 1. Thử hai số 9 và 34. So theo giá trị thì 34 lớn hơn, ra 349. Nhưng 934 mới đúng.

Gợi ý 2. Vậy tiêu chí đúng không phải so hai số, mà so hai **cách nối**.

Gợi ý 3. x đứng trước y khi chuỗi $x + y$ lớn hơn chuỗi $y + x$. Nhưng sorted của Python không nhận hàm so sánh hai tham số, nên bạn phải biến tiêu chí đó thành một **khóa**.

Lời giải. Khóa là chính token lặp lại tới độ dài cố định:

```
KEY = 100
a.sort(key=lambda s: (s * (KEY // len(s) + 1))[:KEY], reverse=True)
```

Cơ sở: $x + y > y + x$ khi và chỉ khi chuỗi lặp vô hạn của x lớn hơn chuỗi lặp vô hạn của y ; hai chuỗi lặp vô hạn nếu khác nhau thì khác nhau trong $\text{lcm}(|x|, |y|)$ ký tự đầu; mọi a_i không quá 10^9 nên không token nào dài quá 10 chữ số, và bội chung nhỏ nhất của hai số không quá 10 thì không vượt 90.

Vì sao không dùng cmp_to_key. Hàm key được gọi n lần, còn hàm so sánh được gọi cỡ $n \log n$ lần và mỗi lần là một lời gọi hàm Python. Đo trên 200000 token của test lớn, sau khi trộn ngẫu nhiên: cmp_to_key mất 0,295 s với 3 258 488 phép so sánh, bản khóa mất 0,061 s. Chênh 4,8 lần, và hai bản cho cùng một chuỗi kết quả.

Và đây là điều đáng nhớ hơn con số trên. Chạy đúng phép đo đó trên test 09 nguyên bản của kho, cmp_to_key chỉ mất 0,025 s với 199 999 phép so sánh. Lý do: test đó **đã gần như sắp sẵn theo chiều giảm**, 199 923 trên 199 999 cặp kề nhau là không tăng, và Timsort nhận ra dãy giảm dài rồi lật lại thay vì sắp thật. Nghĩa là test lớn nhất của bài này không kiểm chi phí hàm so sánh, và bản cmp_to_key vẫn qua được giới hạn 2 giây. Một bộ test cho AC không phải một bằng chứng rằng cách viết của bạn đủ nhanh; nó chỉ là bằng chứng rằng nó đủ nhanh cho **bộ test đó**.

Python. Giữ nguyên bytes từ đầu tới cuối. sys.stdin.buffer.read().split() đã trả về bytes, bytes so sánh theo từng byte nên với chữ số ASCII đó đúng là thứ tự cần, và b"".join(a) tránh giải mã $2 \cdot 10^5$ token sang str không để làm gì.

Bài I. Thuyền cứu hộ

Gợi ý 1. Sắp theo cân nặng rồi nghĩ về người nặng nhất.

Gợi ý 2. Người nặng nhất chắc chắn phải đi trong chuyến này. Câu hỏi duy nhất là có ai đi cùng được không.

Gợi ý 3. Nếu có ai đi cùng được thì người nhẹ nhất là ứng viên tốt nhất.

Lời giải.

```
l, r, boats = 0, n - 1, 0
while l <= r:
    if w[l] + w[r] <= limit:
        l += 1          # the lightest rides along
        r -= 1          # the heaviest always leaves now
    boats += 1
```

Chú ý $r -= 1$ nằm **ngoài** câu điều kiện. Người nặng nhất rồi đi trong mọi trường hợp, hoặc đi một mình hoặc đi cùng người nhẹ nhất. Nếu bạn đưa $r -= 1$ vào trong nhánh if thì khi không ghép được, không chỉ số nào tiến và vòng lặp không bao giờ kết thúc. Ngay ví dụ trong đề đã đủ để lộ: sắp lại là 1 2 2 3, và $1 + 3 = 4$ lớn hơn limit bằng 3.

Đối số đối chỗ. Nếu người nặng nhất ghép được với ai đó, ghép với người nhẹ nhất không bao giờ thiệt: mọi lời giải tối ưu ghép anh ta với người khác đều có thể đổi chỗ hai hành khách nhẹ mà vẫn hợp lệ, vì người nhẹ nhất nhẹ hơn hoặc bằng người kia.

Bài J. Nhảy ít bước nhất

Gợi ý 1. Tập các vị trí tới được sau đúng k bước là một đoạn liên, không phải một tập rời rạc.

Gợi ý 2. Vậy chỉ cần biết đoạn hiện tại kết thúc ở đâu, và đoạn tiếp theo vươn xa tới đâu.

Gợi ý 3. Quét từ trái sang phải, khi đi hết đoạn hiện tại thì tăng số bước và nhảy sang đoạn mới.

Lời giải.

```
for i in range(n):
    if i > reach:
        print(-1)
        return
    if reach >= n - 1:
        break
    f = i + a[i]
    if f > nxt:
        nxt = f
    if i == reach:
        jumps += 1
        reach = nxt
```

Độ phức tạp $O(n)$, không cần tìm kiếm theo chiều rộng.

Điều kiện tăng bước phải đúng lúc i bằng $reach$, tức là đã đi hết tầng hiện tại. Nếu bạn tăng bước ngay khi tìm được một bước nhảy xa hơn thì bạn đếm thừa, vì trong cùng một tầng có thể có nhiều vị trí lần lượt cải thiện nxt .

Đừng quên trường hợp n bằng 1 cho đáp án 0, và trường hợp đường bị cắt cho đáp án -1.

Python. Đây là bài duy nhất của buổi mà phần việc thật là một vòng lặp Python chạy n lượt. Nên thân vòng giữ phẳng: `a[i]` đọc một lần vào biến cục bộ, so sánh viết tay thay cho `max`, và không tách hàm con nào.

Phần IV. Mười code mẫu đầy đủ

Mười file này là **đúng** nội dung trong solutions/ref/, tức đúng thứ bộ chấm chạy. Công tools/qa-note-code.py so từng dòng và trượt nếu hai bên lệch nhau, vì mã in ra giấy không được bộ test nào chấm nên nó là chỗ dễ lạc hậu nhất trong cả kho. Chú thích viết bằng tiếng Anh vì trình duyệt tài liệu không đặt được tiếng Việt có dấu trong khối mã.

Bài A. Chọn nhiều hoạt động nhất

```
# Problem A: pick the largest set of pairwise non overlapping intervals.
# Sort by the RIGHT endpoint, then take an interval whenever it starts at or after the
# endpoint of the last interval taken. Exchange argument: among all optimal answers one
# of them contains the interval that finishes earliest, because swapping the first
# chosen interval for that one never blocks anything it did not already block.
#
# The trap is the sort key, not the loop. Sorting by the LEFT endpoint still compiles,
# still runs, and still prints a plausible number; nothing in the program complains.
#
# Python note. The reader hands back one flat list of tokens, so the two columns come
# out as slices: so[0::2] are the left ends and so[1::2] are the right ends. Slicing and
# zip and sorted all run inside the C library, so there is no Python level loop over the
# 2*10^5 pairs until the greedy sweep itself.
# Complexity: O(n log n).
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    so = list(map(int, data[1:1 + 2 * n]))
    # Pair them as (right, left) so that the plain tuple order already sorts by the
    # right endpoint, which is what the greedy needs.
    v = sorted(zip(so[1::2], so[0::2]))

    cnt = 0
    last = -1          # every l is at least 0, so the first interval is always taken
    for r, l in v:
        if l >= last:
            cnt += 1
            last = r

    sys.stdout.write("%d\n" % cnt)

main()
```

Bài B. Phát bánh

```
# Problem B: serve as many children as possible with the given cookies.
# Sort both sides, then sweep with two indices: give the least demanding unserved child
# the smallest cookie that still satisfies it. Spending a bigger cookie on a small
# demand can never help, and that one sentence is the whole exchange argument.
#
# The trap is the else branch. A cookie too small for the least demanding child left is
# useless for EVERY child left, so only the cookie index moves. Advancing the child
```

```

# index there throws away a child who still had a chance at a later cookie, and the
# answer comes out too small on exactly the inputs where cookies are plentiful.
#
# Python note. sorted(map(int, ...)) builds and sorts the list without a Python level
# loop. Do not write a while loop that calls int() one token at a time here.
# Complexity:  $O(n \log n + m \log m)$ .
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    g = sorted(map(int, data[2:2 + n]))
    s = sorted(map(int, data[2 + n:2 + n + m]))

    i = 0
    j = 0
    served = 0
    while i < n and j < m:
        if s[j] >= g[i]:
            served += 1
            i += 1
            j += 1
        else:
            j += 1          # this cookie is too small for everyone left, drop it

    sys.stdout.write("%d\n" % served)

main()

```

Bài C. Số phòng họp tối thiểu

```

# Problem C: the smallest number of rooms that can host every meeting.
# Turn each meeting into two events and sweep in time order; the answer is the largest
# number of meetings alive at the same instant. A room freed at time t can host a
# meeting that starts at t, so at equal time the END event must be handled BEFORE the
# start event. That tie rule is the entire problem.
#
# Python note, and it is the reason this file does not sort tuples. Python's sort is
# STABLE, while std::sort of C++ is not. A sort keyed on the time alone would resolve
# equal times by the order the events happen to sit in the list, so the program would be
# right or wrong depending on how the list was built rather than on what the sort says,
# and it would keep that answer consistently enough to look correct. The fix is to put
# the tie rule INSIDE the key: pack time and kind into one integer, t*2 for an end and
# t*2+1 for a start. Sorting a list of plain integers is also cheaper than sorting a
# list of tuples, since no tuple objects are built at all.
# Complexity:  $O(n \log n)$ .
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    so = list(map(int, data[1:1 + 2 * n]))
    ev = [x * 2 + 1 for x in so[0::2]]          # start at l: odd, so it sorts AFTER an end
    ev += [x * 2 for x in so[1::2]]           # end at r: even, so it sorts FIRST
    ev.sort()

    cur = 0

```

```

best = 0
for e in ev:
    if e & 1:
        cur += 1
        if cur > best:
            best = cur
    else:
        cur -= 1

sys.stdout.write("%d\n" % best)

main()

```

Bài D. Xếp việc để tổng thời gian hoàn thành nhỏ nhất

```

# Problem D: order the jobs so that the sum of completion times is minimal.
# Shortest processing time first. Exchange argument, written out because this is the
# model problem for it: if a long job of length a sits immediately before a short job of
# length b with a > b, and the pair starts at time T, the pair contributes
# (T + a) + (T + a + b) now and (T + b) + (T + b + a) after the swap. The difference is
# a - b > 0, so the swap strictly lowers the total, and every other job is untouched
# because the pair takes the same total time either way. Hence no optimal order contains
# such a pair, so the optimal order is ascending.
#
# Python note, two of them.
# 1. The answer reaches about 7*10^14 on the large test. In C++ that is one of the
#    graded traps of the problem and needs long long. Python integers never overflow so
#    that trap does not exist in this branch. The trap that DOES exist is writing / when
#    // is meant: / always returns a float and floats stop being exact past about
#    9*10^15, which is inside this range.
# 2. sum(accumulate(t)) computes every prefix sum and adds them up inside the C library.
#    There is no Python level loop in this solution at all.
# Complexity: O(n log n).
import sys
from itertools import accumulate

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    t = sorted(map(int, data[1:1 + n]))    # ascending: the shortest job goes first
    sys.stdout.write("%d\n" % sum(accumulate(t)))

main()

```

Bài E. Đổ xăng ít lần nhất

```

# Problem E: fewest refuelling stops driving from 0 to D on a full tank worth C.
# Drive as far as the tank allows. When the next station is out of reach, refuel at the
# FARTHEST station already passed, because that one pushes the reachable point out the
# most. Postponing a stop is never worse, which is the exchange argument once more.
#
# The inner while must drain EVERY station within reach. Turning it into an if means
# refuelling at each station in turn: the program still runs, still terminates, and

```

```

# still prints a plausible count, just a much larger one. Index i never moves back, so
# the two nested loops together are O(n) after the sort.
#
# Python note. D goes up to 10^18. In C++ every position variable has to be 64 bit or
# the answer is silently wrong; Python integers are arbitrary precision, so the only way
# to lose here is to let a float in through / instead of //.
# Complexity: O(n log n), dominated by the sort.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    D = int(data[0]); C = int(data[1]); n = int(data[2])
    x = sorted(map(int, data[3:3 + n]))

    reach = C
    stops = 0
    i = 0
    while reach < D:
        best = -1
        while i < n and x[i] <= reach:
            best = x[i]          # keep going: the last one drained is the farthest
            i += 1
        if best < 0:
            sys.stdout.write("-1\n")    # no station within reach, the trip is stuck
            return
        stops += 1
        reach = best + C

    sys.stdout.write("%d\n" % stops)

main()

```

Bài F. Chọn việc có hạn chót

```

# Problem F: unit length jobs with deadlines, maximise the total profit.
# Taking the most profitable jobs first is WRONG: one fat job with a late deadline can
# crowd out two jobs that both only fit early. The correct shape is the REGRET greedy.
# Walk the jobs in deadline order, always take the job, and keep a min heap of the
# profits taken so far. The moment the number taken exceeds the current deadline, give
# back the smallest profit in the heap. Giving back the smallest is never worse than
# giving back any other, which is the exchange argument in its running form.
#
# Python note, and it is a reversal worth remembering. heapq is a MIN heap, which is
# exactly what this problem wants, so the default is already right. C++ is the other way
# round: priority_queue is a MAX heap and needs the three argument form with greater<>
# to become a min heap. When a Python greedy wants the LARGEST element instead, there is
# no max heap in the module: push -value and negate again on the way out.
#
# Second Python note. heappop(taken) is not taken.pop(). The heap invariant only puts
# the minimum at index 0; the last slot of the list holds no particular element, so
# list.pop gives back an arbitrary profit and quietly breaks the heap as well.
# Complexity: O(n log n).
import sys
from heapq import heappush, heappop

def main():
    data = sys.stdin.buffer.read().split()

```

```

n = int(data[0])
so = list(map(int, data[1:1 + 2 * n]))
job = sorted(zip(so[0::2], so[1::2])) # (deadline, profit), deadline ascending

taken = []
total = 0
for d, p in job:
    heappush(taken, p) # take it first, decide later
    total += p
    if len(taken) > d: # one job too many for this deadline
        total -= heappop(taken) # regret the least profitable choice

sys.stdout.write("%d\n" % total)

main()

```

Bài G. Phủ đoạn bằng ít khoảng nhất

```

# Problem G: cover the segment [0, M] with as few of the given intervals as possible.
# Stand at the point already covered. Among all intervals that start at or before that
# point, take the one reaching FARTHEST, then repeat from the new frontier. Keeping the
# last candidate seen, or the first one that merely fits, is the classic wrong version:
# it produces a larger count on some inputs and the right count on the small ones.
#
# The loop must also be able to give up. If no interval pushes the frontier forward
# there is a real gap and the answer is -1; without that test the loop spins forever on
# unsolvable data, which shows up as a timeout rather than as a wrong answer.
#
# Python note. Index i never moves back, so the nested while is O(n) in total. Reading
# v[i][0] and v[i][1] twice per step is cheaper here than unpacking into a tuple, and
# the comparison is written out instead of calling max() because a call into max() in a
# hot loop costs more than the comparison it performs.
# Complexity: O(n log n), dominated by the sort.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    M = int(data[0]); n = int(data[1])
    so = list(map(int, data[2:2 + 2 * n]))
    v = sorted(zip(so[0::2], so[1::2])) # (left, right), left ascending

    covered = 0
    used = 0
    i = 0
    while covered < M:
        best = covered
        while i < n and v[i][0] <= covered:
            r = v[i][1]
            if r > best:
                best = r
            i += 1
        if best == covered: # nothing reaches past the frontier
            sys.stdout.write("-1\n")
            return
        covered = best
        used += 1

    sys.stdout.write("%d\n" % used)

```

```
main()
```

Bài H. Ghép thành số lớn nhất

```
# Problem H: concatenate the numbers into the largest possible string.
# The order is "x comes before y when x+y is larger than y+x read as text". Comparing by
# numeric value or by length is wrong: 9 against 34 gives 934, not 349.
#
# Python note, and this is the whole point of the problem in this language. sorted() has
# no comparator argument. The way to use one is functools.cmp_to_key, and then every
# single comparison becomes a Python level function call plus a wrapper object. A KEY is
# the cheap route: a key function is called n times, a comparator O(n log n) times
# through the interpreter. Both versions were measured on the 2*10^5 test of this
# problem, and the numbers are in section I.6 of the note.
#
# The key is the token repeated out to a fixed length. That works because x+y > y+x
# holds exactly when the infinite repetition of x is larger than that of y, and two such
# repetitions, if they differ at all, differ inside the first lcm(len x, len y)
# characters. Every a_i is at most 10^9, so no token is longer than 10 digits and no lcm
# of two lengths at most 10 exceeds 90.
#
# Everything stays in bytes. The reader already returns bytes, bytes compare byte by
# byte which for ASCII digits is exactly the order wanted, and b"".join avoids decoding
# 2*10^5 tokens into str objects for nothing.
# Complexity: O(n log n) comparisons of fixed length keys.
import sys

KEY = 100      # at least lcm(i, j) for any two token lengths i, j at most 10, that is 90

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = data[1:1 + n]
    a.sort(key=lambda s: (s * (KEY // len(s) + 1))[:KEY], reverse=True)
    sys.stdout.buffer.write(b"".join(a) + b"\n")

main()
```

Bài I. Thuyền cứu hộ

```
# Problem I: fewest boats, at most two people each, total weight at most limit.
# Sort, then look at the heaviest person left. That person boards a boat now whatever
# happens; the only open question is whether the lightest person left fits alongside.
# Exchange argument: if the heaviest can share with anybody, sharing with the lightest
# is never worse, because the lightest weighs no more than that other passenger and so
# every optimal pairing can be rewritten to use the lightest.
#
# The decrement of the right index therefore sits OUTSIDE the if. Moving it inside is
# not merely a wrong count: on the first pair that does not fit, neither index moves and
# the loop never ends. The scoreboard reports that as a timeout, so the symptom looks
# nothing like the cause.
# Complexity: O(n log n), dominated by the sort.
```



```
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); limit = int(data[1])
    w = sorted(map(int, data[2:2 + n]))

    l = 0
    r = n - 1
    boats = 0
    while l <= r:
        if w[l] + w[r] <= limit:
            l += 1          # the lightest rides along
            r -= 1          # the heaviest always leaves now
            boats += 1

    sys.stdout.write("%d\n" % boats)

main()
```

Bài J. Nhảy ít bước nhất

```
# Problem J: fewest jumps from position 1 to position n.
# Think in layers. Everything reachable in exactly k jumps forms a CONTIGUOUS block, so
# there is no need to know which positions are reachable, only where the block ends. The
# next block ends at the farthest index any position of the current block can open up.
# One left to right pass, O(n), and no breadth first search.
#
# The counter goes up only when i has walked to the END of the current layer, that is
# when i == reach. Bumping it as soon as a farther jump appears overcounts, because
# several positions inside one layer can improve the frontier one after another.
#
# Two edge cases the problem leans on: n equal to 1 answers 0, and a cut road, where the
# sweep reaches an index beyond the frontier, answers -1.
#
# Python note. This is the one solution of the topic whose work really is a Python level
# loop over n, so it keeps the loop body flat: no max() call, no helper function, and
# # a[i] read once into a local.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))

    reach = 0          # farthest index reachable with the jumps used so far
    nxt = 0             # farthest index the current layer can open up
    jumps = 0
    for i in range(n):
        if i > reach:
            sys.stdout.write("-1\n")      # the road is cut here
            return
        if reach >= n - 1:
            break
        f = i + a[i]
        if f > nxt:
            nxt = f
```

```
        if i == reach:                # current layer exhausted, step to the next
            jumps += 1
            reach = nxt

    sys.stdout.write("%d\n" % jumps)

main()
```